

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра прикладной математики

Курсовой проект по курсу
«УРАВНЕНИЯ МАТЕМАТИЧЕСКОЙ ФИЗИКИ»

Группа

ПМ-35

Студентка

ЦЫБЕНОВА МАРИНА
БАТО-МУНКОВНА

Новосибирск

2026

1. Задание

Гиперболическая задача с эллиптическим оператором. Неявная трехслойная схема по времени.

МКЭ для двумерной краевой задачи для эллиптического уравнения в цилиндрической (r, z) системе координат. Базисные функции квадратичные на треугольниках. Краевые условия всех типов. Коэффициент диффузии λ разложить по линейным базисным функциям. Матрицу СЛАУ генерировать на разреженном строчном формате. Для решения СЛАУ использовать МСГ или ЛОС с неполной факторизацией.

2. Постановка задачи

Гиперболическая задача с эллиптическим оператором в цилиндрической системе координат (r, z) . Искомая функция $u = u(r, z, t)$ удовлетворяет уравнению

$$\chi \frac{\partial^2 u}{\partial t^2} + \sigma \frac{\partial u}{\partial t} - \operatorname{div}(\lambda \operatorname{grad} u) = f$$
$$\chi \frac{\partial^2 u}{\partial t^2} + \sigma \frac{\partial u}{\partial t} - \frac{1}{r} \frac{\partial}{\partial r} \left(r \lambda \frac{\partial u}{\partial r} \right) - \frac{\partial}{\partial z} \left(\lambda \frac{\partial u}{\partial z} \right) = f$$

заданным в некоторой области D с границей $S = S_1 \cup S_2 \cup S_3$ и краевыми условиями

$$u|_{S_1} = u_g(r, z, t)$$
$$\lambda \frac{\partial u}{\partial n} \Big|_{S_2} = \theta(r, z, t)$$
$$\lambda \frac{\partial u}{\partial n} \Big|_{S_3} + \beta(u|_{S_3} - u_\beta(r, z, t)) = 0$$

в которых $u|_{S_i}$ - значение искомой функции u на границе S_i , а $\frac{\partial u}{\partial n} \Big|_{S_i}$ - значение на S_i производной функции u по направлению внешней нормали к поверхности S_i , $i = 1, 2, 3$.

Также для гиперболической задачи задаются два начальных условия:

$u(r, z, 0) = u_0(r, z)$ - начальное распределение функции,

$\frac{\partial u}{\partial t}(r, z, 0) = v_0(r, z)$ - начальная скорость.

3. Вариационная постановка

Преобразуем начальное уравнение

$$\int_{\Omega} \chi \frac{\partial^2 u}{\partial t^2} v r dr dz + \int_{\Omega} \sigma \frac{\partial u}{\partial t} v r dr dz + \int_{\Omega} \lambda \left(\frac{\partial u}{\partial r} \frac{\partial v}{\partial r} + \frac{\partial u}{\partial z} \frac{\partial v}{\partial z} \right) r dr dz + \int_{S_3} \beta u v r dS$$
$$= \int_{\Omega} f v r dr dz + \int_{S_2} \theta v r dS + \int_{S_3} \beta u_\beta v r dS$$

4. Дискретизация по времени

Введем сетку по времени $t_0, t_1, \dots, t_{k-2}, t_{k-1}, t_k, \dots$

Рассмотрим промежуток $[t_{k-2}, t_k]$.

На этом промежутке приближенно представим функцию $u(r, z, t)$ через значения на трех временных слоях с помощью квадратичных полиномов Лагранжа:

$$u(r, z, t) = u^{k-2}(r, z)\eta_2^k(t) + u^{k-1}(r, z)\eta_1^k(t) + u^k(r, z)\eta_0^k(t)$$

здесь $u^{k-2} = u(r, z, t_{k-2})$, $u^{k-1} = u(r, z, t_{k-1})$, $u^k = u(r, z, t_k)$.

Введем обозначения:

$$\Delta t = t_k - t_{k-2}, \Delta t_1 = t_{k-1} - t_{k-2}, \Delta t_0 = t_k - t_{k-1} \Rightarrow \Delta t = \Delta t_0 + \Delta t_1.$$

Для трехслойной неявной схемы необходимо задать значения решения на двух первых временных слоях. Значения на нулевом слое находятся из начального состояния:

$$q_i^0 = u_0(r_i, z_i).$$

Также в узлах вычисляется начальная скорость:

$$v_i^0 = v_0(r_i, z_i).$$

Первый слой находится по формуле

$$q_i^1 = q^0 + \Delta t_0 v^0 + \frac{\Delta t_0^2}{2} a^0.$$

Ускорение находится из системы:

$$Ma^0 = \frac{1}{\chi} (b^0 - Gq^0 - \sigma Mv^0).$$

Базисные квадратичные полиномы Лагранжа имеют вид:

$$\begin{aligned}\eta_2^k(t) &= \frac{(t - t_{k-1})(t - t_k)}{\Delta t_1 \Delta t}, \\ \eta_1^k(t) &= -\frac{(t - t_{k-2})(t - t_k)}{\Delta t_1 \Delta t_0}, \\ \eta_0^k(t) &= \frac{(t - t_{k-2})(t - t_{k-1})}{\Delta t_0 \Delta t}.\end{aligned}$$

Они равны единице на своем временном слое и нулю на остальных двух.

Так как схема является неявной, производные вычисляются на новом временном слое $t = t_k$.

$$\begin{aligned}\left. \frac{\partial \eta_2^k}{\partial t} \right|_{t=t_k} &= \frac{\Delta t_0}{\Delta t_1 \Delta t}, \\ \left. \frac{\partial \eta_1^k}{\partial t} \right|_{t=t_k} &= -\frac{\Delta t}{\Delta t_1 \Delta t_0}, \\ \left. \frac{\partial \eta_0^k}{\partial t} \right|_{t=t_k} &= \frac{\Delta t + \Delta t_0}{\Delta t \Delta t_0}, \\ \left. \frac{\partial^2 \eta_2^k}{\partial t^2} \right|_{t=t_k} &= \frac{2}{\Delta t_1 \Delta t}, \\ \left. \frac{\partial^2 \eta_1^k}{\partial t^2} \right|_{t=t_k} &= -\frac{2}{\Delta t_1 \Delta t_0},\end{aligned}$$

$$\left. \frac{\partial^2 \eta_0^k}{\partial t^2} \right|_{t=t_k} = \frac{2}{\Delta t \Delta t_0}.$$

Перепишем начальное уравнение:

$$\chi \left(\frac{2}{\Delta t_1 \Delta t} u^{k-2} - \frac{2}{\Delta t_1 \Delta t_0} u^{k-1} + \frac{2}{\Delta t \Delta t_0} u^k \right) + \sigma \left(\frac{\Delta t_0}{\Delta t_1 \Delta t} u^{k-2} - \frac{\Delta t}{\Delta t_1 \Delta t_0} u^{k-1} + \frac{\Delta t + \Delta t_0}{\Delta t \Delta t_0} u^k \right) - \frac{1}{r} \frac{\partial}{\partial r} \left(r \lambda \frac{\partial u^k}{\partial r} \right) - \frac{\partial}{\partial z} \left(\lambda \frac{\partial u^k}{\partial z} \right) = f^k$$

Выполним конечноэлементную аппроксимацию краевой задачи для получившегося уравнения. При выполнении постановки Галеркина получим:

$$A_{ij} = \int_{\Omega} \lambda \left(\frac{\partial \psi_j}{\partial r} \frac{\partial \psi_i}{\partial r} + \frac{\partial \psi_j}{\partial z} \frac{\partial \psi_i}{\partial z} \right) r d\Omega + \left(\frac{2\chi}{\Delta t \Delta t_0} + \frac{\sigma(\Delta t + \Delta t_0)}{\Delta t \Delta t_0} \right) \int_{\Omega} \psi_j \psi_i r d\Omega + \int_{S_3} \beta \psi_j \psi_i r dS$$

$$b_i = \int_{\Omega} f^k \psi_i r d\Omega - \left(\frac{2\chi}{\Delta t_1 \Delta t} + \frac{\sigma \Delta t_0}{\Delta t_1 \Delta t} \right) \sum_{j=1}^n q_j^{k-2} \int_{\Omega} \psi_j \psi_i r d\Omega$$

$$+ \left(\frac{2\chi}{\Delta t_1 \Delta t_0} + \frac{\sigma \Delta t}{\Delta t_1 \Delta t_0} \right) \sum_{j=1}^n q_j^{k-1} \int_{\Omega} \psi_j \psi_i r d\Omega + \int_{S_2} \theta \psi_i r dS + \int_{S_3} \beta u_{\beta} \psi_i r dS$$

где принимая $G_{ij} = \int_{\Omega} \lambda \left(\frac{\partial \psi_j}{\partial r} \frac{\partial \psi_i}{\partial r} + \frac{\partial \psi_j}{\partial z} \frac{\partial \psi_i}{\partial z} \right) r dr dz$, $M_{ij}^{\chi} = \int_{\Omega} \chi \psi_j \psi_i r dr dz$, $M_{ij}^{\sigma} = \int_{\Omega} \sigma \psi_j \psi_i r dr dz$, $M_{ij}^{S_3} = \int_{S_3} \beta \psi_j \psi_i r dS$ и $b_i^k = \int_{\Omega} f^k \psi_i r d\Omega + \int_{S_2} \theta \psi_i r dS + \int_{S_3} \beta u_{\beta} \psi_i r dS$ получаем

$$\left(\frac{2}{\Delta t \Delta t_0} M^{\chi} + \frac{\Delta t + \Delta t_0}{\Delta t \Delta t_0} M^{\sigma} + G + M^{S_3} \right) q^k = b^k - \left(\frac{2}{\Delta t_1 \Delta t} M^{\chi} + \frac{\Delta t_0}{\Delta t_1 \Delta t} M^{\sigma} \right) q^{k-2} + \left(\frac{2}{\Delta t_1 \Delta t_0} M^{\chi} + \frac{\Delta t}{\Delta t_1 \Delta t_0} M^{\sigma} \right) q^{k-1}$$

где M - матрица массы, G - матрица жесткости, M^{S_3} - матрица вкладов от краевых условий третьего рода, b^k - вектор правой части, построенный по значениям функции f и параметром θ и βu_{β} краевых условий на текущем временном (k -м) слое, q^{k-2} и q^{k-1} - решение на двух предыдущих слоях по времени, q^k - искомое решение на текущем слое по времени.

5. Базисные функции

Рассмотрим один треугольный конечный элемент Ω_m сетки в плоскости (r, z) с вершинами (r_1^m, z_1^m) , (r_2^m, z_2^m) , (r_3^m, z_3^m) .

На нем вводятся L-координаты (координаты площади) L , удовлетворяющие

$$L_1 + L_2 + L_3 = 1,$$

$$L_i(r_j^m, z_j^m) = \delta_{ij}, i, j = 1, 2, 3.$$

Базисные функции квадратичного треугольного элемента удобно выразить через L -координаты

$$\psi_1(r, z) = L_1(2L_1 - 1)$$

$$\psi_2(r, z) = L_2(2L_2 - 1)$$

$$\psi_3(r, z) = L_3(2L_3 - 1)$$

$$\begin{aligned}\psi_4(r, z) &= 4L_1L_2 \\ \psi_5(r, z) &= 4L_2L_3 \\ \psi_6(r, z) &= 4L_3L_1\end{aligned}$$

Здесь первые три базисные функции соответствуют узлам в вершинах треугольника, а последние три – узлам в серединах его сторон.

Частные производные базисных функций по координатам (r, z) получаются дифференцированием этих выражений с учётом того, что L_i являются линейными функциями:

$$\begin{aligned}L_i(r, z) &= \alpha_0^i + \alpha_1^i r + \alpha_2^i z, \\ \frac{\partial L_i}{\partial r} &= \alpha_1^i, \quad \frac{\partial L_i}{\partial z} = \alpha_2^i. \\ \frac{\partial \psi_1}{\partial r} &= (4L_1 - 1) \alpha_1^1, \quad \frac{\partial \psi_1}{\partial z} = (4L_1 - 1) \alpha_2^1, \\ \frac{\partial \psi_2}{\partial r} &= (4L_2 - 1) \alpha_1^2, \quad \frac{\partial \psi_2}{\partial z} = (4L_2 - 1) \alpha_2^2, \\ \frac{\partial \psi_3}{\partial r} &= (4L_3 - 1) \alpha_1^3, \quad \frac{\partial \psi_3}{\partial z} = (4L_3 - 1) \alpha_2^3, \\ \frac{\partial \psi_4}{\partial r} &= 4(L_2 \alpha_1^1 + L_1 \alpha_1^2), \quad \frac{\partial \psi_4}{\partial z} = 4(L_2 \alpha_2^1 + L_1 \alpha_2^2), \\ \frac{\partial \psi_5}{\partial r} &= 4(L_3 \alpha_1^2 + L_2 \alpha_1^3), \quad \frac{\partial \psi_5}{\partial z} = 4(L_3 \alpha_2^2 + L_2 \alpha_2^3), \\ \frac{\partial \psi_6}{\partial r} &= 4(L_1 \alpha_1^3 + L_3 \alpha_1^1), \quad \frac{\partial \psi_6}{\partial z} = 4(L_1 \alpha_2^3 + L_3 \alpha_2^1).\end{aligned}$$

6. Вычисление локальных матриц

Поскольку в варианте прописано, что λ нужно разложить по линейным базисным функциям:

$$\lambda = \lambda_1 L_1 + \lambda_2 L_2 + \lambda_3 L_3$$

Матрица жесткости тогда будет иметь вид:

$$\begin{aligned}\widehat{G}_{ij} &= \int_{\Omega} (\sum_{k=1}^3 \lambda_k L_k) \left(\frac{\partial \widehat{\Psi}_j}{\partial r} \frac{\partial \widehat{\Psi}_i}{\partial r} + \frac{\partial \widehat{\Psi}_j}{\partial z} \frac{\partial \widehat{\Psi}_i}{\partial z} \right) r dr dz = \sum_{k=1}^3 \lambda_k \widehat{R}_{ij}, \\ \widehat{R}_{ij} &= \int_{\Omega} L_k \left(\frac{\partial \widehat{\Psi}_j}{\partial r} \frac{\partial \widehat{\Psi}_i}{\partial r} + \frac{\partial \widehat{\Psi}_j}{\partial z} \frac{\partial \widehat{\Psi}_i}{\partial z} \right) r dr dz.\end{aligned}$$

Матрица массы:

$$\widehat{M}_{ij} = \int_{\Omega} \widehat{\Psi}_j \widehat{\Psi}_i r dr dz.$$

Вектор правой части:

$$\widehat{b}_i = \int_{\Omega} f \widehat{\Psi}_i r dr dz.$$

где $f = \sum_{j=1}^6 f_j \psi_j$. f_j – значения функции в узлах квадратичного конечного элемента.

Вычислим элемент b_1

$$\begin{aligned}
b_1 &= \int_{\Omega_k} f \psi_1 dr dz = \int_{\Omega_k} \psi_1 \sum_{j=1}^6 f_j \psi_j dr dz = \\
&= \int_{\Omega_k} f_1 \psi_1 \psi_1 + f_2 \psi_2 \psi_1 + f_3 \psi_3 \psi_1 + f_4 \psi_4 \psi_1 + f_5 \psi_5 \psi_1 + f_6 \psi_6 \psi_1 dx dy = \\
&= \frac{|\det D|}{360} (6f_1 - f_2 - f_3 - 4f_5)
\end{aligned}$$

Тогда остальные элементы вектора правой части будут иметь вид:

$$\begin{aligned}
b_2 &= \frac{|\det D|}{360} (-f_1 + 6f_2 - f_3 - 4f_6) \\
b_3 &= \frac{|\det D|}{360} (-f_1 - f_2 + 6f_3 - 4f_4) \\
b_4 &= \frac{|\det D|}{360} (-4f_3 + 32f_4 + 16f_5 + 16f_6) \\
b_5 &= \frac{|\det D|}{360} (-4f_1 + 16f_4 + 32f_5 + 16f_6) \\
b_6 &= \frac{|\det D|}{360} (-4f_2 + 16f_4 + 16f_5 + 32f_6)
\end{aligned}$$

Элементы глобальной матрицы:

При формировании глобальной матрицы из локальных, полученных суммированием соответствующих матриц массы и жесткости, учитываем соответствие локальной и глобальной нумераций каждого конечного элемента. Глобальная нумерация каждого конечного элемента однозначно определяет позиции вклада его локальной матрицы в глобальную. Поэтому, зная глобальные номера соответствующих узлов конечного элемента, определяем и то, какие элементы глобальной матрицы изменятся при учете текущего конечного элемента. Аналогичным образом определяется вклад локального вектора правой части в глобальный. При учете текущего локального вектора изменятся те элементы глобального вектора правой части, номера которых совпадают с глобальными номерами узлов, присутствующих в этом конечном элементе.

7. Учет краевых условий

Учет первых краевых условий проводится после полной сборки глобальной матрицы и правой части (включая учет условий второго и третьего рода). В глобальной матрице и глобальном векторе находим соответствующую глобальному номеру краевого узла строку, и ставим вместо диагонального элемента глобальной матрицы на этой строке единицу и нули вместо внедиагональных элементов, а вместо элемента с таким номером в вектор правой части присваиваем значения первого краевого условия.

Структуры данных для хранения информации о краевых условиях организованы вектором целых чисел, где сначала указано количество рёбер, затем первое число - номер узла начала ребра, второе – номер узла конца ребра, а третье – номер формулы, которая будет вычислять значение параметра u_g .

Краевые условия второго рода на рёбрах $S_{i,j}$ вносит вклад только в правую часть СЛАУ.

$$\hat{b}^{s_2} = \frac{h_m}{30} \begin{pmatrix} 4 & 2 & -1 \\ 2 & 16 & 2 \\ -1 & 2 & 4 \end{pmatrix} \begin{pmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{pmatrix}$$

При учете третьих краевых условий формируются локальная матрица и вектор правой части, которые заносятся в СЛАУ аналогично локальной матрице конечного элемента и локального вектора правой части конечного элемента.

$$\hat{b}^{s_3} = \frac{\beta h_m}{30} \begin{pmatrix} 4 & 2 & -1 \\ 2 & 16 & 2 \\ -1 & 2 & 4 \end{pmatrix} \begin{pmatrix} u_{\beta_1} \\ u_{\beta_2} \\ u_{\beta_3} \end{pmatrix}$$

$$A^{s_3} = \frac{\beta h_m}{30} \begin{pmatrix} 4 & 2 & -1 \\ 2 & 16 & 2 \\ -1 & 2 & 4 \end{pmatrix}$$

Где h_m - длина ребра.

8. Описание разработанной программы

Программа реализована на языке C++ и предназначена для численного решения гиперболической задачи с эллиптическим оператором методом конечных элементов в цилиндрической системе координат (r,z). Для аппроксимации по пространству используются квадратичные базисные функции на треугольных конечных элементах, поэтому каждый треугольник задаётся шестью узлами: тремя вершинами и тремя серединами сторон. По времени реализована неявная трёхслойная схема. В программе предусмотрена работа как с равномерной, так и с неравномерной сеткой по времени.

Основные данные программы хранятся в следующих структурах:

Node — координаты узла сетки (r,z);

Triangle — конечный элемент, содержащий номера шести узлов и номер области;

S1Node — узлы с краевыми условиями первого рода;

Edge — ребра с краевыми условиями второго и третьего рода;

matrix — разреженная матрица, хранящая диагональ и внедиагональные элементы в строчном формате.

Матрица СЛАУ хранится в разреженном строчном формате. Для этого используются массивы ig, jg, di, ggl, ggu. Массив di хранит диагональные элементы, ggl — нижний треугольник матрицы, ggu — верхний треугольник, а массивы ig и jg задают структуру ненулевых элементов. Портрет матрицы формируется по конечноэлементной сетке, это выполняется в функции generatePortrait.

Считывание исходных данных выполняется в функции readData. Из файлов считываются координаты узлов, номера узлов в треугольниках, номера областей, а также данные о краевых условиях первого, второго и третьего рода. Параметры итерационного метода, то есть максимальное число итераций и точность, считываются из файла solver.txt.

Сборка глобальных матриц выполняется в функции assemble. В ней для каждого конечного элемента вычисляются локальная матрица жесткости, локальная матрица массы и локальный вектор правой части. Интегралы по треугольнику вычисляются численным интегрированием по семи точкам. Коэффициент λ аппроксимируется линейно через значения в вершинах треугольника. После вычисления локальные вклады добавляются в глобальные матрицы и правую часть.

Краевые условия учитываются отдельными функциями. Краевые условия второго рода добавляют вклад только в правую часть. Краевые условия третьего рода добавляют вклад и в матрицу, и в правую часть. Эти условия обрабатываются в функции applyNaturalBC.

Краевые условия первого рода учитываются после сборки системы: соответствующая строка матрицы изменяется так, чтобы значение решения в граничном узле совпадало с заданным значением. Это выполняется в функции `applyDirichletBC`.

Так как для гиперболической задачи требуется два начальных условия, сначала формируются два первых временных слоя. Нулевой слой q^0 задаётся из начального условия $u_0(r,z)$. Первый слой q^1 вычисляется по формуле Тейлора с использованием начальной скорости $v_0(r,z)$ и начального ускорения. Для нахождения ускорения решается вспомогательная система с матрицей массы. Это реализовано в функции `firstLayers`.

В программе предусмотрена возможность работы как с равномерной, так и с неравномерной сеткой по времени. Для этого используется функция `generateTimeGrid()`. В зависимости от значения флага строится равномерная или неравномерная сетка. Равномерная сетка строится по формуле $t_i = \frac{iT}{N_t}$. В неравномерной сетке относительные веса шагов чередуются $\Delta t_i = 1 - \alpha, \Delta t_{i+1} = 1 + \alpha$. Затем все шаги умножаются на нормирующий множитель так, чтобы последний узел совпадал с T .

Далее выполняется основной цикл по времени. На каждом временном слое собираются матрица жесткости G , матрица массы M и вектор правой части b . Затем формируется итоговая система:

$$Aq^k = b,$$

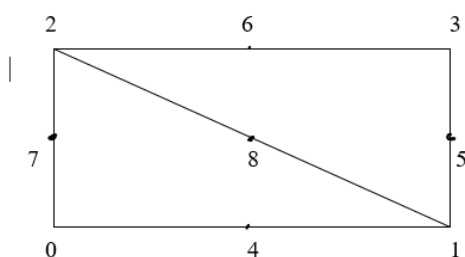
где матрица A состоит из матрицы жесткости и матрицы массы с коэффициентом, зависящим от шага по времени. Правая часть содержит вклад функции f , а также решения с двух предыдущих временных слоев q^{k-1} и q^{k-2} . В программе это реализовано через коэффициенты `coefA`, `coefPrev`, `coefOld`.

Для решения системы линейных алгебраических уравнений используется метод сопряжённых градиентов с неполной LU-факторизацией в качестве предобуславливателя. Умножение матрицы на вектор выполняется функцией `multA`, неполная факторизация — функцией `calcLU`, а сам итерационный процесс — функцией `doMSG_LU`.

Для проверки работы программы используется файл `tests.cpp`, в котором задаются точное решение, коэффициенты уравнения, правая часть, краевые и начальные условия. В процессе расчета программа выводит таблицу сравнения точного и численного решения в узлах сетки, а также максимальную абсолютную погрешность.

9. Тестирование

1. Тесты на порядок аппроксимации по времени на равномерной сетке



Узлы и их координаты	Конечные элементы
0: 1.0 0.0	0: 0 1 2 4 8 7 1: 1 3 2 5 6 8
1: 3.0 0.0	
2: 1.0 1.0	
3: 3.0 1.0	
4: 2.0 0.0	
5: 3.0 0.5	
6: 2.0 1.0	

Проверка временной схемы на
линейной зависимости по времени.

7: 1.0 0.5

8: 2.0 0.5

$$u(r, z) = (r^2 + z)t; \lambda = 1;$$

$$f = r^2 + z - 4t$$

t = 0.25					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	2.5000000000E-01	2.5000000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	2.2500000000E+00	2.2500000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	5.0000000000E-01	5.0000000000E-01	0.000000E+00
3	3.000000E+00	1.000000E+00	2.5000000000E+00	2.5000000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	2.3750000000E+00	2.3750000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	1.2500000000E+00	1.2500000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	3.7500000000E-01	3.7500000000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	1.1250000000E+00	1.1250000000E+00	1.332300E-15
t = 0.5					
0	1.000000E+00	0.000000E+00	5.0000000000E-01	5.0000000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	4.5000000000E+00	4.5000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	4.7500000000E+00	4.7500000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	2.5000000000E+00	2.5000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	7.5000000000E-01	7.5000000000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	2.2500000000E+00	2.2500000000E+00	1.554300E-14
t = 0.75					
0	1.000000E+00	0.000000E+00	7.5000000000E-01	7.5000000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	6.7500000000E+00	6.7500000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	1.5000000000E+00	1.5000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	7.5000000000E+00	7.5000000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	3.0000000000E+00	3.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	7.1250000000E+00	7.1250000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	3.7500000000E+00	3.7500000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.1250000000E+00	1.1250000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	3.3750000000E+00	3.3750000000E+00	6.172800E-14
t = 1					
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.5000000000E+00	1.154600E-13

Вывод: Численное решение совпадает с аналитическим.

Проверка временной схемы на квадратичной зависимости по времени.

$$u(r, z) = (r^2 + z)t^2; \lambda = 1; f = 2(r^2 + z) + 2(r^2 + z)t - 4t^2;$$

t = 0.25					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	6.2500000000E-02	6.2500000000E-02	0.000000E+00
1	3.000000E+00	0.000000E+00	5.6250000000E-01	5.6250000000E-01	0.000000E+00
2	1.000000E+00	1.000000E+00	1.2500000000E-01	1.2500000000E-01	0.000000E+00
3	3.000000E+00	1.000000E+00	6.2500000000E-01	6.2500000000E-01	0.000000E+00
4	2.000000E+00	0.000000E+00	2.5000000000E-01	2.5000000000E-01	0.000000E+00
5	3.000000E+00	5.000000E-01	5.9375000000E-01	5.9375000000E-01	0.000000E+00
6	2.000000E+00	1.000000E+00	3.1250000000E-01	3.1250000000E-01	0.000000E+00
7	1.000000E+00	5.000000E-01	9.3750000000E-02	9.3750000000E-02	0.000000E+00
8	2.000000E+00	5.000000E-01	2.8125000000E-01	2.8125000000E-01	7.216400E-16
t = 0.5					
0	1.000000E+00	0.000000E+00	2.5000000000E-01	2.5000000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	2.2500000000E+00	2.2500000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	5.0000000000E-01	5.0000000000E-01	0.000000E+00
3	3.000000E+00	1.000000E+00	2.5000000000E+00	2.5000000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	2.3750000000E+00	2.3750000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	1.2500000000E+00	1.2500000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	3.7500000000E-01	3.7500000000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	1.1250000000E+00	1.1250000000E+00	8.215700E-15
t = 0.75					
0	1.000000E+00	0.000000E+00	5.6250000000E-01	5.6250000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	5.0625000000E+00	5.0625000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	1.1250000000E+00	1.1250000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	5.6250000000E+00	5.6250000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	2.2500000000E+00	2.2500000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	5.3437500000E+00	5.3437500000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	2.8125000000E+00	2.8125000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	8.4375000000E-01	8.4375000000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	2.5312500000E+00	2.5312500000E+00	1.509900E-14
t = 1					
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.5000000000E+00	3.108600E-14

Вывод: Численное решение совпадает с аналитическим.

Проверка временной схемы на кубической зависимости по времени.

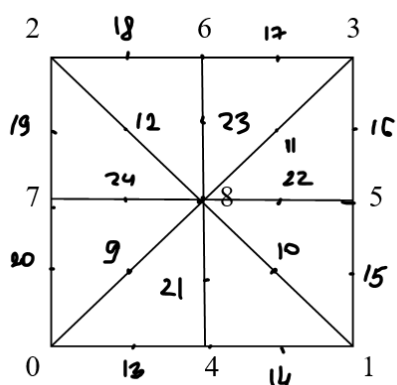
$$u(r, z) = (r^2 + z)t^3; \lambda = 1; f = 6t(r^2 + z) + 3t^2(r^2 + z) - 4t^3;$$

t = 0.25					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	1.5625000000E-02	1.5625000000E-02	0.000000E+00
1	3.000000E+00	0.000000E+00	1.4062500000E-01	1.4062500000E-01	0.000000E+00

2	1.000000E+00	1.000000E+00	3.1250000000E-02	3.1250000000E-02	0.000000E+00
3	3.000000E+00	1.000000E+00	1.5625000000E-01	1.5625000000E-01	0.000000E+00
4	2.000000E+00	0.000000E+00	6.2500000000E-02	6.2500000000E-02	0.000000E+00
5	3.000000E+00	5.000000E-01	1.4843750000E-01	1.4843750000E-01	0.000000E+00
6	2.000000E+00	1.000000E+00	7.8125000000E-02	7.8125000000E-02	0.000000E+00
7	1.000000E+00	5.000000E-01	2.3437500000E-02	2.3437500000E-02	0.000000E+00
8	2.000000E+00	5.000000E-01	7.0312500000E-02	1.0228405740E-01	3.197200E-02
t = 0.5					
0	1.000000E+00	0.000000E+00	1.2500000000E-01	1.2500000000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	1.1250000000E+00	1.1250000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.5000000000E-01	2.5000000000E-01	0.000000E+00
3	3.000000E+00	1.000000E+00	1.2500000000E+00	1.2500000000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	5.0000000000E-01	5.0000000000E-01	0.000000E+00
5	3.000000E+00	5.000000E-01	1.1875000000E+00	1.1875000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	6.2500000000E-01	6.2500000000E-01	0.000000E+00
7	1.000000E+00	5.000000E-01	1.8750000000E-01	1.8750000000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	5.6250000000E-01	6.5594259420E-01	9.344300E-02
t = 0.75					
0	1.000000E+00	0.000000E+00	4.2187500000E-01	4.2187500000E-01	0.000000E+00
1	3.000000E+00	0.000000E+00	3.7968750000E+00	3.7968750000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	8.4375000000E-01	8.4375000000E-01	0.000000E+00
3	3.000000E+00	1.000000E+00	4.2187500000E+00	4.2187500000E+00	0.000000E+00
4	2.000000E+00	0.000000E+00	1.6875000000E+00	1.6875000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	4.0078125000E+00	4.0078125000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	2.1093750000E+00	2.1093750000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	6.3281250000E-01	6.3281250000E-01	0.000000E+00
8	2.000000E+00	5.000000E-01	1.8984375000E+00	2.0205218760E+00	1.220800E-01
t = 1					
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.6028133486E+00	1.028100E-01

Вывод: На внутреннем узле появляется заметная погрешность. Можно сделать вывод, что порядок аппроксимации равен 2.

2. Тестирование на порядок сходимости по времени на неполиномиальной функции на равномерной сетке.



Узлы и их координаты	Конечные элементы
0: 0 0	0: 4 0 8 13 9 21
1: 1 0	1: 1 4 8 14 21 10
2: 0 1	2: 1 8 5 10 22 15
3: 1 1	3: 8 3 5 11 16 22

$u(r, z) = (r^2 + z)\sin(t); \quad \lambda = 1; f = (r^2 + z)\cos t - (r^2 + z + 4)\sin t$. Пространственная сетка фиксировалась, а шаг по времени уменьшался. Рассмотрим на момент времени $t = 1$.

4: 0.5 0.0	4: 8 6 3 23 17 11
5: 1 0.5	5: 8 2 6 12 18 23
6: 0.5 1	6: 7 2 8 19 12 24
7: 0 0.5	7: 0 7 8 20 24 9
8: 0.5 0.5	
9: 0.25 0.25	
10: 0.75 0.25	
11: 0.75 0.75	
12: 0.25 0.75	
13: 0.25 0.0	
14: 0.75 0.0	
15: 1.0 0.25	
16: 1.0 0.75	
17: 0.75 1.0	
18: 0.25 1.0	
19: 0 0.75	
20: 0 0.25	
21: 0.5 0.25	
22: 0.75 0.5	
23: 0.5 0.75	
24: 0.25 0.5	

$\tau = 0.1$					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.042800E-13
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.042800E-13
3	1.000000E+00	1.000000E+00	1.6829419696E+00	1.6829419696E+00	-4.085600E-13
4	5.000000E-01	0.000000E+00	2.1036774620E-01	2.1036774620E-01	-5.107000E-14
5	1.000000E+00	5.000000E-01	1.2622064772E+00	1.2622064772E+00	-3.064200E-13
6	5.000000E-01	1.000000E+00	1.0518387310E+00	1.0518387310E+00	-2.553500E-13
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-1.021400E-13
8	5.000000E-01	5.000000E-01	6.3110323860E-01	6.2378221930E-01	-7.321000E-03
9	2.500000E-01	2.500000E-01	2.6295968280E-01	2.5794009860E-01	-5.019600E-03
10	7.500000E-01	2.500000E-01	6.8369517520E-01	6.8041618430E-01	-3.279000E-03
11	7.500000E-01	7.500000E-01	1.1044306676E+00	1.1005173101E+00	-3.913400E-03
12	2.500000E-01	7.500000E-01	6.8369517520E-01	6.7792330050E-01	-5.771900E-03
13	2.500000E-01	0.000000E+00	5.2591936600E-02	5.2591936600E-02	-1.276800E-14
14	7.500000E-01	0.000000E+00	4.7332742900E-01	4.7332742900E-01	-1.149100E-13
15	1.000000E+00	2.500000E-01	1.0518387310E+00	1.0518387310E+00	-2.553500E-13
16	1.000000E+00	7.500000E-01	1.4725742234E+00	1.4725742234E+00	-3.572700E-13
17	7.500000E-01	1.000000E+00	1.3147984138E+00	1.3147984138E+00	-3.188600E-13
18	2.500000E-01	1.000000E+00	8.9406292140E-01	8.9406292140E-01	-2.169400E-13

19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-1.532100E-13
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-5.107000E-14
21	5.000000E-01	2.500000E-01	4.2073549240E-01	4.1593204410E-01	-4.803400E-03
22	7.500000E-01	5.000000E-01	8.9406292140E-01	8.8921512280E-01	-4.847800E-03
23	5.000000E-01	7.500000E-01	8.4147098480E-01	8.3579870820E-01	-5.672300E-03
24	2.500000E-01	5.000000E-01	4.7332742900E-01	4.6634436810E-01	-6.983100E-03
$\tau = 0.05$					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	8.4147098480E-01	8.4147098480E-01	-3.058200E-11
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-3.058200E-11
3	1.000000E+00	1.000000E+00	1.6829419696E+00	1.6829419696E+00	-6.116300E-11
4	5.000000E-01	0.000000E+00	2.1036774620E-01	2.1036774620E-01	-7.645400E-12
5	1.000000E+00	5.000000E-01	1.2622064772E+00	1.2622064772E+00	-4.587300E-11
6	5.000000E-01	1.000000E+00	1.0518387310E+00	1.0518387310E+00	-3.822700E-11
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-1.529100E-11
8	5.000000E-01	5.000000E-01	6.3110323860E-01	6.2767495190E-01	-3.428300E-03
9	2.500000E-01	2.500000E-01	2.6295968280E-01	2.6030088590E-01	-2.658800E-03
10	7.500000E-01	2.500000E-01	6.8369517520E-01	6.8214756600E-01	-1.547600E-03
11	7.500000E-01	7.500000E-01	1.1044306676E+00	1.1026385303E+00	-1.792100E-03
12	2.500000E-01	7.500000E-01	6.8369517520E-01	6.8081602790E-01	-2.879100E-03
13	2.500000E-01	0.000000E+00	5.2591936600E-02	5.2591936500E-02	-1.911400E-12
14	7.500000E-01	0.000000E+00	4.7332742900E-01	4.7332742890E-01	-1.720200E-11
15	1.000000E+00	2.500000E-01	1.0518387310E+00	1.0518387310E+00	-3.822700E-11
16	1.000000E+00	7.500000E-01	1.4725742234E+00	1.4725742234E+00	-5.351800E-11
17	7.500000E-01	1.000000E+00	1.3147984138E+00	1.3147984137E+00	-4.778400E-11
18	2.500000E-01	1.000000E+00	8.9406292140E-01	8.9406292130E-01	-3.249300E-11
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-2.293600E-11
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-7.645400E-12
21	5.000000E-01	2.500000E-01	4.2073549240E-01	4.1836647120E-01	-2.369000E-03
22	7.500000E-01	5.000000E-01	8.9406292140E-01	8.9184074560E-01	-2.222200E-03
23	5.000000E-01	7.500000E-01	8.4147098480E-01	8.3880682540E-01	-2.664200E-03
24	2.500000E-01	5.000000E-01	4.7332742900E-01	4.6976676160E-01	-3.560700E-03
$\tau = 0.025$					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	8.4147098480E-01	8.4147098480E-01	5.111500E-13
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.111500E-13
3	1.000000E+00	1.000000E+00	1.6829419696E+00	1.6829419696E+00	1.022300E-12
4	5.000000E-01	0.000000E+00	2.1036774620E-01	2.1036774620E-01	1.277900E-13
5	1.000000E+00	5.000000E-01	1.2622064772E+00	1.2622064772E+00	7.667200E-13
6	5.000000E-01	1.000000E+00	1.0518387310E+00	1.0518387310E+00	6.388200E-13
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	2.555700E-13
8	5.000000E-01	5.000000E-01	6.3110323860E-01	6.2949361280E-01	-1.609600E-03
9	2.500000E-01	2.500000E-01	2.6295968280E-01	2.6162711020E-01	-1.332600E-03
10	7.500000E-01	2.500000E-01	6.8369517520E-01	6.8295855150E-01	-7.366200E-04
11	7.500000E-01	7.500000E-01	1.1044306676E+00	1.1035866591E+00	-8.440100E-04
12	2.500000E-01	7.500000E-01	6.8369517520E-01	6.8228827380E-01	-1.406900E-03
13	2.500000E-01	0.000000E+00	5.2591936600E-02	5.2591936600E-02	3.194700E-14
14	7.500000E-01	0.000000E+00	4.7332742900E-01	4.7332742900E-01	2.874400E-13
15	1.000000E+00	2.500000E-01	1.0518387310E+00	1.0518387310E+00	6.388200E-13
16	1.000000E+00	7.500000E-01	1.4725742234E+00	1.4725742234E+00	8.944000E-13
17	7.500000E-01	1.000000E+00	1.3147984138E+00	1.3147984138E+00	7.986900E-13
18	2.500000E-01	1.000000E+00	8.9406292140E-01	8.9406292140E-01	5.430100E-13

19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	3.833600E-13
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	1.277900E-13
21	5.000000E-01	2.500000E-01	4.2073549240E-01	4.1958667950E-01	-1.148800E-03
22	7.500000E-01	5.000000E-01	8.9406292140E-01	8.9302375780E-01	-1.039200E-03
23	5.000000E-01	7.500000E-01	8.4147098480E-01	8.4020569570E-01	-1.265300E-03
24	2.500000E-01	5.000000E-01	4.7332742900E-01	4.7157637160E-01	-1.751100E-03
$\tau = 0.0125$					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	8.4147098480E-01	8.4147098480E-01	3.341800E-12
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	3.341800E-12
3	1.000000E+00	1.000000E+00	1.6829419696E+00	1.6829419696E+00	6.683500E-12
4	5.000000E-01	0.000000E+00	2.1036774620E-01	2.1036774620E-01	8.354400E-13
5	1.000000E+00	5.000000E-01	1.2622064772E+00	1.2622064772E+00	5.012700E-12
6	5.000000E-01	1.000000E+00	1.0518387310E+00	1.0518387310E+00	4.177300E-12
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	1.670900E-12
8	5.000000E-01	5.000000E-01	6.3110323860E-01	6.3032352490E-01	-7.797100E-04
9	2.500000E-01	2.500000E-01	2.6295968280E-01	2.6230355960E-01	-6.561200E-04
10	7.500000E-01	2.500000E-01	6.8369517520E-01	6.8333905070E-01	-3.561200E-04
11	7.500000E-01	7.500000E-01	1.1044306676E+00	1.1040240023E+00	-4.066700E-04
12	2.500000E-01	7.500000E-01	6.8369517520E-01	6.8300715580E-01	-6.880200E-04
13	2.500000E-01	0.000000E+00	5.2591936600E-02	5.2591936600E-02	2.088600E-13
14	7.500000E-01	0.000000E+00	4.7332742900E-01	4.7332742900E-01	1.879700E-12
15	1.000000E+00	2.500000E-01	1.0518387310E+00	1.0518387310E+00	4.177300E-12
16	1.000000E+00	7.500000E-01	1.4725742234E+00	1.4725742234E+00	5.848000E-12
17	7.500000E-01	1.000000E+00	1.3147984138E+00	1.3147984138E+00	5.221600E-12
18	2.500000E-01	1.000000E+00	8.9406292140E-01	8.9406292140E-01	3.550600E-12
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	2.506300E-12
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	8.354400E-13
21	5.000000E-01	2.500000E-01	4.2073549240E-01	4.2017376000E-01	-5.617300E-04
22	7.500000E-01	5.000000E-01	8.9406292140E-01	8.9356487620E-01	-4.980500E-04
23	5.000000E-01	7.500000E-01	8.4147098480E-01	8.4085747940E-01	-6.135100E-04
24	2.500000E-01	5.000000E-01	4.7332742900E-01	4.7246895000E-01	-8.584800E-04

Шаг по времени	E	E_i/E_{i+1}	Порядок
h	-1.90E-03	-	-
h/2	-9.25E-04	2.06E+00	1.042043861
h/4	-4.45E-04	2.08E+00	1.054277389
h/8	-2.17E-04	2.05E+00	1.039047377

Вывод: При уменьшении шага по времени в два раза ошибка также уменьшается примерно в два раза. Полученный порядок сходимости близок к первому, что соответствует реализованной неявной трёхслойной схеме.

3. Тестирование на порядок аппроксимации на неравномерной сетке

$u(r, z) = (r^2 + z)t$					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00

3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.5000000000E+00	3.552700E-15

$u(r, z) = (r^2 + z)t^2;$					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.5000000000E+00	-2.664500E-15

$u(r, z) = (r^2 + z)t^3;$					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	1.000000E+00	0.000000E+00	1.0000000000E+00	1.0000000000E+00	0.000000E+00
1	3.000000E+00	0.000000E+00	9.0000000000E+00	9.0000000000E+00	0.000000E+00
2	1.000000E+00	1.000000E+00	2.0000000000E+00	2.0000000000E+00	0.000000E+00
3	3.000000E+00	1.000000E+00	1.0000000000E+01	1.0000000000E+01	0.000000E+00
4	2.000000E+00	0.000000E+00	4.0000000000E+00	4.0000000000E+00	0.000000E+00
5	3.000000E+00	5.000000E-01	9.5000000000E+00	9.5000000000E+00	0.000000E+00
6	2.000000E+00	1.000000E+00	5.0000000000E+00	5.0000000000E+00	0.000000E+00
7	1.000000E+00	5.000000E-01	1.5000000000E+00	1.5000000000E+00	0.000000E+00
8	2.000000E+00	5.000000E-01	4.5000000000E+00	4.9175491723E+00	4.175500E-01

Вывод: также как и на равномерной сетке, порядок равен 2.

4. Тестирование на порядок сходимости на неравномерной сетке

dt = 0.1					
Номер узла	r	z	Ожидаемое значение	Полученное значение	Погрешность
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	8.4147098480E-01	8.4147098480E-01	-1.537500E-11
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-1.537500E-11
3	1.000000E+00	1.000000E+00	1.6829419696E+00	1.6829419696E+00	-3.075000E-11
4	5.000000E-01	0.000000E+00	2.1036774620E-01	2.1036774620E-01	-3.843800E-12
5	1.000000E+00	5.000000E-01	1.2622064772E+00	1.2622064772E+00	-2.306300E-11
6	5.000000E-01	1.000000E+00	1.0518387310E+00	1.0518387310E+00	-1.921900E-11
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-7.687600E-12
8	5.000000E-01	5.000000E-01	6.3110323860E-01	6.1721920240E-01	-1.388400E-02
9	2.500000E-01	2.500000E-01	2.6295968280E-01	2.5465835030E-01	-8.301300E-03
10	7.500000E-01	2.500000E-01	6.8369517520E-01	6.7705323970E-01	-6.641900E-03

11	7.500000E-01	7.500000E-01	1.1044306676E+00	1.0960010119E+00	-8.429700E-03
12	2.500000E-01	7.500000E-01	6.8369517520E-01	6.7308155700E-01	-1.061400E-02
13	2.500000E-01	0.000000E+00	5.2591936600E-02	5.2591936500E-02	-9.609500E-13
14	7.500000E-01	0.000000E+00	4.7332742900E-01	4.7332742890E-01	-8.648500E-12
15	1.000000E+00	2.500000E-01	1.0518387310E+00	1.0518387310E+00	-1.921900E-11
16	1.000000E+00	7.500000E-01	1.4725742234E+00	1.4725742234E+00	-2.690600E-11
17	7.500000E-01	1.000000E+00	1.3147984138E+00	1.3147984137E+00	-2.402400E-11
18	2.500000E-01	1.000000E+00	8.9406292140E-01	8.9406292130E-01	-1.633600E-11
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-1.153100E-11
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-3.843800E-12
21	5.000000E-01	2.500000E-01	4.2073549240E-01	4.1194210710E-01	-8.793400E-03
22	7.500000E-01	5.000000E-01	8.9406292140E-01	8.8394187230E-01	-1.012100E-02
23	5.000000E-01	7.500000E-01	8.4147098480E-01	8.3009908680E-01	-1.137200E-02
24	2.500000E-01	5.000000E-01	4.7332742900E-01	4.6121940920E-01	-1.210800E-02
dt = 0.05					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-5.593300E-13
3	1.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-5.593300E-13
4	5.000000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
5	1.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-2.796700E-13
6	5.000000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-5.593300E-13
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-2.796700E-13
8	5.000000E-01	5.000000E-01	4.2073549240E-01	4.1644735260E-01	-4.288100E-03
9	2.500000E-01	2.500000E-01	2.1036774620E-01	2.0737212220E-01	-2.995600E-03
10	7.500000E-01	2.500000E-01	2.1036774620E-01	2.0877336430E-01	-1.594400E-03
11	7.500000E-01	7.500000E-01	6.3110323860E-01	6.2887449720E-01	-2.228700E-03
12	2.500000E-01	7.500000E-01	6.3110323860E-01	6.2736911060E-01	-3.734100E-03
13	2.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
14	7.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
15	1.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-1.398300E-13
16	1.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-4.195500E-13
17	7.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-5.593300E-13
18	2.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-5.593300E-13
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-4.195500E-13
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-1.398300E-13
21	5.000000E-01	2.500000E-01	2.1036774620E-01	2.0772678540E-01	-2.641000E-03
22	7.500000E-01	5.000000E-01	4.2073549240E-01	4.1816707520E-01	-2.568400E-03
23	5.000000E-01	7.500000E-01	6.3110323860E-01	6.2760252110E-01	-3.500700E-03
24	2.500000E-01	5.000000E-01	4.2073549240E-01	4.1640245050E-01	-4.333000E-03
dt = 0.025					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.936200E-11
3	1.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.936200E-11
4	5.000000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
5	1.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-1.468100E-11
6	5.000000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.936200E-11
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	-1.468100E-11
8	5.000000E-01	5.000000E-01	4.2073549240E-01	4.1869835440E-01	-2.037100E-03
9	2.500000E-01	2.500000E-01	2.1036774620E-01	2.0880957930E-01	-1.558200E-03
10	7.500000E-01	2.500000E-01	2.1036774620E-01	2.0958695060E-01	-7.808000E-04

11	7.500000E-01	7.500000E-01	6.3110323860E-01	6.3007309180E-01	-1.030100E-03
12	2.500000E-01	7.500000E-01	6.3110323860E-01	6.2931648800E-01	-1.786800E-03
13	2.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
14	7.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
15	1.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-7.340500E-12
16	1.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-2.202100E-11
17	7.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.936200E-11
18	2.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	-2.936200E-11
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	-2.202100E-11
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	-7.340500E-12
21	5.000000E-01	2.500000E-01	2.1036774620E-01	2.0903823610E-01	-1.329500E-03
22	7.500000E-01	5.000000E-01	4.2073549240E-01	4.1953002300E-01	-1.205500E-03
23	5.000000E-01	7.500000E-01	6.3110323860E-01	6.2947060340E-01	-1.632600E-03
24	2.500000E-01	5.000000E-01	4.2073549240E-01	4.1859179370E-01	-2.143700E-03
dt = 0.0125					
0	0.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
1	1.000000E+00	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
2	0.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.251400E-14
3	1.000000E+00	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.251400E-14
4	5.000000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
5	1.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	2.625700E-14
6	5.000000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.251400E-14
7	0.000000E+00	5.000000E-01	4.2073549240E-01	4.2073549240E-01	2.625700E-14
8	5.000000E-01	5.000000E-01	4.2073549240E-01	4.1976198340E-01	-9.735100E-04
9	2.500000E-01	2.500000E-01	2.1036774620E-01	2.0958641230E-01	-7.813300E-04
10	7.500000E-01	2.500000E-01	2.1036774620E-01	2.0998788800E-01	-3.798600E-04
11	7.500000E-01	7.500000E-01	6.3110323860E-01	6.3061344550E-01	-4.897900E-04
12	2.500000E-01	7.500000E-01	6.3110323860E-01	6.3024109410E-01	-8.621400E-04
13	2.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
14	7.500000E-01	0.000000E+00	0.0000000000E+00	0.0000000000E+00	0.000000E+00
15	1.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	1.312800E-14
16	1.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	3.941300E-14
17	7.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.251400E-14
18	2.500000E-01	1.000000E+00	8.4147098480E-01	8.4147098480E-01	5.251400E-14
19	0.000000E+00	7.500000E-01	6.3110323860E-01	6.3110323860E-01	3.941300E-14
20	0.000000E+00	2.500000E-01	2.1036774620E-01	2.1036774620E-01	1.312800E-14
21	5.000000E-01	2.500000E-01	2.1036774620E-01	2.0971165350E-01	-6.560900E-04
22	7.500000E-01	5.000000E-01	4.2073549240E-01	4.2016146440E-01	-5.740300E-04
23	5.000000E-01	7.500000E-01	6.3110323860E-01	6.3032528690E-01	-7.779500E-04
24	2.500000E-01	5.000000E-01	4.2073549240E-01	4.1968691010E-01	-1.048600E-03

Шаг по времени	E	E_i/E_{i+1}	Порядок
h	-5.40E-04	-	-
h/2	-2.62E-04	2.06E+00	1.045328562
h/4	-1.28E-04	2.04E+00	1.029707573
h/8	-6.34E-05	2.02E+00	1.016807756

Вывод: для неравномерной сетки порядок аппроксимации также близок к единице.

10. Программа

Main.cpp

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <vector>
#include <cmath>
#include <string>
#include "header.h"

using namespace std;

std::vector<Node> nodes;
std::vector<Triangle> triangles;
std::vector<S1Node> s1_nodes;
std::vector<Edge> s2_edges;
std::vector<Edge> s3_edges;

void printTable(double t, const vector<double> &q)
{
    int N = (int)nodes.size();

    cout << "  таблица для времени t = " << fixed << setprecision(6) << t << "\n";
    cout << string(95, '=') << "\n";
    cout << setw(6) << "Node"
        << setw(10) << "r"
        << setw(10) << "z"
        << setw(18) << "Exact"
        << setw(18) << "Obtained"
        << setw(12) << "Diff" << "\n";

    double maxErr = 0.0;

    for (int i = 0; i < N; i++) {
        double ex = u_exact(nodes[i].r, nodes[i].z, t);
        double sol = q[i];
        double diff = sol - ex;

        if (std::fabs(diff) > maxErr) {
            maxErr = std::fabs(diff);
        }

        cout << fixed << setprecision(6)
            << setw(6) << i
            << setw(10) << nodes[i].r
            << setw(10) << nodes[i].z
            << setprecision(10)
            << setw(18) << ex
            << setw(18) << sol
            << scientific << setprecision(4)
            << setw(12) << diff
            << fixed << "\n";
    }

    cout << string(95, '=') << "\n";
    cout << "Max Absolute Error at t = "
        << setprecision(6) << t
        << ": " << scientific << maxErr << "\n\n";
}

double calcMaxError(double t, const vector<double> &q)
{
    double maxErr = 0.0;

    for (int i = 0; i < (int)nodes.size(); i++) {
        double ex = u_exact(nodes[i].r, nodes[i].z, t);
```

```

        double diff = std::fabs(q[i] - ex);

        if (diff > maxErr) {
            maxErr = diff;
        }
    }

    return maxErr;
}

vector<double> generateTimeGrid(double T, int steps, bool nonUniform, double alpha =
0.35)
{
    vector<double> time(steps + 1, 0.0);

    if (!nonUniform) { //равномерная сетка
        for (int i = 0; i <= steps; i++) {
            time[i] = T * i / steps;
        }

        return time;
    }

    vector<double> dtRaw(steps, 0.0);
    double sum = 0.0;

    for (int i = 0; i < steps; i++) { // неравномерная сетка
        double factor;

        if (i % 2 == 0) {
            factor = 1.0 - alpha;
        }
        else {
            factor = 1.0 + alpha;
        }

        dtRaw[i] = factor;
        sum += dtRaw[i];
    }

    double scale = T / sum;

    for (int i = 0; i < steps; i++) {
        time[i + 1] = time[i] + dtRaw[i] * scale;
    }

    time[steps] = T;

    return time;
}

void readData(int &maxIter, double &eps)
{
    std::ifstream in;

    in.open("nodes.txt");
    int nNodes;

    in >> nNodes;

    nodes.resize(nNodes);

    for (int i = 0; i < nNodes; i++) {
        in >> nodes[i].r >> nodes[i].z;
    }
}

```

```

in.close();

in.open("triangles.txt");

int nTri;

in >> nTri;

triangles.resize(nTri);

for (int i = 0; i < nTri; i++) {
    for (int j = 0; j < 6; j++) {
        in >> triangles[i].n[j];
    }

    in >> triangles[i].region;
}

in.close();

auto readBC = [] (const char *f, auto &vec, bool isS1) {
    std::ifstream fin(f);

    if (!fin) {
        return;
    }

    int c;

    fin >> c;

    vec.resize(c);

    for (int i = 0; i < c; i++) {
        if (isS1) {
            fin >> ((S1Node &)vec[i]).node
                >> ((S1Node &)vec[i]).funcNum;
        }
        else {
            fin >> ((Edge &)vec[i]).node1
                >> ((Edge &)vec[i]).node2
                >> ((Edge &)vec[i]).funcNum;
        }
    }

    fin.close();
};

readBC("s1_nodes.txt", s1_nodes, true);
readBC("s2_edges.txt", s2_edges, false);
readBC("s3_edges.txt", s3_edges, false);

in.open("solver.txt");

if (in) {
    in >> maxIter >> eps;
}
else {
    maxIter = 1000;
    eps = 1e-13;
}

in.close();
}

```

```

void setBc1Vector(vector<double> &q, double t)
{
    for (const auto &bc : s1_nodes) {
        int idx = bc.node;

        q[idx] = u_s1(bc.funcNum, nodes[idx].r, nodes[idx].z, t);
    }
}

void setBc1SystemByValues(matrix &A, vector<double> &b, const vector<double> &val)
{
    for (const auto &bc : s1_nodes) {
        int idx = bc.node;
        double v = val[idx];

        A.di[idx] = 1.0;
        b[idx] = v;

        for (int k = A.ig[idx]; k < A.ig[idx + 1]; k++) {
            int j = A.jg[k];

            b[j] -= A.ggu[k] * v;

            A.ggl[k] = 0.0;
            A.ggu[k] = 0.0;
        }

        for (int i = idx + 1; i < A.n; i++) {
            for (int k = A.ig[i]; k < A.ig[i + 1]; k++) {
                if (A.jg[k] == idx) {
                    b[i] -= A.ggl[k] * v;

                    A.ggl[k] = 0.0;
                    A.ggu[k] = 0.0;
                }
            }
        }
    }
}

void firstLayers(double tau, double chi, double sigma, int maxIter, double eps,
vector<double> &q0, vector<double> &q1)
{
    int N = (int)nodes.size();

    q0.assign(N, 0.0);
    q1.assign(N, 0.0);

    vector<double> v0(N, 0.0);
    vector<double> a0_bc(N, 0.0);

    for (int i = 0; i < N; i++) {
        double r = nodes[i].r;
        double z = nodes[i].z;

        q0[i] = u0_f(r, z);
        v0[i] = v0_f(r, z);
        a0_bc[i] = a0_f(r, z);
    }

    setBc1Vector(q0, 0.0);

    matrix G0, M0;

```

```

generatePortrait(G0);
generatePortrait(M0);

vector<double> b0(N, 0.0);
vector<double> Gq0(N, 0.0);
vector<double> Mv0(N, 0.0);
vector<double> bAcc(N, 0.0);
vector<double> a0(N, 0.0);

assemble(G0, M0, b0, 0.0);

applyNaturalBC(G0, b0, 0.0);

multA(G0, q0, Gq0);
multA(M0, v0, Mv0);

for (int i = 0; i < N; i++) {
    bAcc[i] = (b0[i] - Gq0[i] - sigma * Mv0[i]) / chi;
}

setBc1SystemByValues(M0, bAcc, a0_bc);

msg_lu msg0;

doMSG_LU(M0, msg0, a0, bAcc, maxIter, eps);

for (int i = 0; i < N; i++) {
    q1[i] = q0[i] + tau * v0[i] + 0.5 * tau * tau * a0[i];
}

setBc1Vector(q1, tau);
}

int main()
{
    setlocale(0, "");
    int maxIter;
    double eps;

    readData(maxIter, eps);

    double T = 1.0;
    double baseTau = 0.00625;
    bool nonUniform = true;
    double nonUniformAlpha = 0.35;

    double chi = 1.0;
    double sigma = 1.0;

    int steps = (int)std::round(T / baseTau);

    vector<double> timeGrid = generateTimeGrid(T, steps, nonUniform, nonUniformAlpha);

    int N = (int)nodes.size();

    vector<double> q_old(N, 0.0);
    vector<double> q_prev(N, 0.0);
    vector<double> q(N, 0.0);

    vector<double> b(N, 0.0);
    vector<double> rs(N, 0.0);

    vector<double> Mq_old(N, 0.0);
    vector<double> Mq_prev(N, 0.0);

```

```

double tauFirst = timeGrid[1] - timeGrid[0];

firstLayers(tauFirst, chi, sigma, maxIter, eps, q_old, q_prev);

printTable(0.0, q_old);

matrix G, M, A;

generatePortrait(G);
generatePortrait(M);
generatePortrait(A);

msg_lu msg;

cout << "Starting time loop..." << endl;

for (int k = 2; k <= steps; k++) {
    double t = timeGrid[k];

    double dt0 = timeGrid[k] - timeGrid[k - 1];
    double dt1 = timeGrid[k - 1] - timeGrid[k - 2];
    double dt = dt0 + dt1;

    double coefA =
        2.0 * chi / (dt * dt0)
        +
        sigma * (dt + dt0) / (dt * dt0);

    double coefPrev =
        2.0 * chi / (dt1 * dt0)
        +
        sigma * dt / (dt1 * dt0);

    double coefOld =
        2.0 * chi / (dt1 * dt)
        +
        sigma * dt0 / (dt1 * dt);

    assemble(G, M, b, t);

    for (int i = 0; i < N; i++) {
        A.di[i] = G.di[i] + coefA * M.di[i];
    }

    for (int i = 0; i < (int)A.jg.size(); i++) {
        A.ggl[i] = G.ggl[i] + coefA * M.ggl[i];
        A.ggu[i] = G.ggu[i] + coefA * M.ggu[i];
    }

    multA(M, q_prev, Mq_prev);
    multA(M, q_old, Mq_old);

    for (int i = 0; i < N; i++) {
        rs[i] = b[i] + coefPrev * Mq_prev[i] - coefOld * Mq_old[i];
    }

    applyNaturalBC(A, rs, t);
    applyDirichletBC(A, rs, t);

    q.assign(N, 0.0);

    doMSG_LU(A, msg, q, rs, maxIter, eps);

    if (std::fabs(t - 0.25) < 1e-7 ||
        std::fabs(t - 0.50) < 1e-7 ||

```

```

        std::fabs(t - 0.75) < 1e-7 ||
        std::fabs(t - 1.00) < 1e-7) {
            printTable(t, q);
        }

        q_old = q_prev;
        q_prev = q;
    }

    double finalError = calcMaxError(T, q_prev);
    return 0;
}

```

Solver.cpp

```

#include "header.h"
#include <cmath>
#include <algorithm>
#include <iostream>

bool findEdgeMidNode(int n1, int n2, int &mid) // поиск среднего узла ребра
{
    for (const auto &t : triangles) {
        int v0 = t.n[0];
        int v1 = t.n[1];
        int v2 = t.n[2];

        if ((n1 == v0 && n2 == v1) || (n1 == v1 && n2 == v0)) {
            mid = t.n[3];
            return true;
        }

        if ((n1 == v1 && n2 == v2) || (n1 == v2 && n2 == v1)) {
            mid = t.n[4];
            return true;
        }

        if ((n1 == v2 && n2 == v0) || (n1 == v0 && n2 == v2)) {
            mid = t.n[5];
            return true;
        }
    }

    return false;
}

```



```

void generatePortrait(matrix &A) // генерация портрета
{
    int n = (int)nodes.size();

    A.n = n;

    std::vector<std::vector<int>> adj(n);

    for (const auto &t : triangles) {
        for (int i = 0; i < 6; i++) {
            for (int j = 0; j < i; j++) {
                int r = t.n[i];
                int c = t.n[j];

                if (r == c) {
                    continue;
                }

                if (r < c) {
                    std::swap(r, c);
                }

                adj[r].push_back(c);
            }
        }
    }

    A.ig.resize(n + 1);
    A.ig[0] = 0;

    for (int i = 0; i < n; i++) {
        auto &v = adj[i];

        std::sort(v.begin(), v.end());
        v.erase(std::unique(v.begin(), v.end()), v.end());

        A.ig[i + 1] = A.ig[i] + (int)v.size();
    }

    int nnz = A.ig[n];

```

```

A.jg.resize(nnz);
A.di.assign(n, 0.0);
A.ggl.assign(nnz, 0.0);
A.ggu.assign(nnz, 0.0);

```

```

int k = 0;

```

```

for (int i = 0; i < n; i++) {
    for (int c : adj[i]) {
        A.jg[k++] = c;
    }
}

```

```

void addElement(matrix &A, std::vector<double> &b, const double locM[6][6], const
double locB[6], const int idx[6])

```

```

{
    for (int i = 0; i < 6; i++) {
        int row = idx[i];

        A.di[row] += locM[i][i];
        b[row] += locB[i];

        for (int j = 0; j < i; j++) {
            int col = idx[j];

            int r = row;
            int c = col;

            if (r < c) {
                std::swap(r, c);
            }

            int pos = A.ig[r];

            while (pos < A.ig[r + 1] && A.jg[pos] != c) {
                pos++;
            }

            if (pos < A.ig[r + 1]) {
                A.ggl[pos] += locM[i][j];
            }
        }
    }
}

```

```

        A.ggu[pos] += locM[j][i];
    }
}
}

```

```

void getShapeFuncs(double L1, double L2, double L3, double N[6], double dN_dL[6][3])
{
    N[0] = L1 * (2.0 * L1 - 1.0);
    N[1] = L2 * (2.0 * L2 - 1.0);
    N[2] = L3 * (2.0 * L3 - 1.0);
    N[3] = 4.0 * L1 * L2;
    N[4] = 4.0 * L2 * L3;
    N[5] = 4.0 * L3 * L1;

    dN_dL[0][0] = 4.0 * L1 - 1.0;
    dN_dL[0][1] = 0.0;
    dN_dL[0][2] = 0.0;

    dN_dL[1][0] = 0.0;
    dN_dL[1][1] = 4.0 * L2 - 1.0;
    dN_dL[1][2] = 0.0;

    dN_dL[2][0] = 0.0;
    dN_dL[2][1] = 0.0;
    dN_dL[2][2] = 4.0 * L3 - 1.0;

    dN_dL[3][0] = 4.0 * L2;
    dN_dL[3][1] = 4.0 * L1;
    dN_dL[3][2] = 0.0;

    dN_dL[4][0] = 0.0;
    dN_dL[4][1] = 4.0 * L3;
    dN_dL[4][2] = 4.0 * L2;

    dN_dL[5][0] = 4.0 * L3;
    dN_dL[5][1] = 0.0;
    dN_dL[5][2] = 4.0 * L1;
}

```

```

void assemble(matrix &G, matrix &M, std::vector<double> &rs, double t)

```

```

{
    std::fill(G.di.begin(), G.di.end(), 0.0);
    std::fill(G.ggl.begin(), G.ggl.end(), 0.0);
    std::fill(G.ggu.begin(), G.ggu.end(), 0.0);

    std::fill(M.di.begin(), M.di.end(), 0.0);
    std::fill(M.ggl.begin(), M.ggl.end(), 0.0);
    std::fill(M.ggu.begin(), M.ggu.end(), 0.0);

    std::fill(rs.begin(), rs.end(), 0.0);

    const double qa = 0.797426985353087;
    const double qb = 0.101286507323456;

    const double qc = 0.059715871789770;
    const double qd = 0.470142064105115;

    const double w1 = 0.225;
    const double w2 = 0.125939180544827;
    const double w3 = 0.132394152788527;

    const double L_QUAD[7][3] = {
        {1.0 / 3.0, 1.0 / 3.0, 1.0 / 3.0},
        {qa, qb, qb},
        {qb, qa, qb},
        {qb, qb, qa},
        {qc, qd, qd},
        {qd, qc, qd},
        {qd, qd, qc}
    };

    const double W_QUAD[7] = {
        w1, w2, w2, w2, w3, w3, w3
    };

    for (const auto &tri : triangles) {
        double locG[6][6] = { 0.0 };
        double locM[6][6] = { 0.0 };
        double locB[6] = { 0.0 };

        double R[6];
    }
}

```

```

double Z[6];

for (int i = 0; i < 6; i++) {
    R[i] = nodes[tri.n[i]].r;
    Z[i] = nodes[tri.n[i]].z;
}

double detJ =
    (R[1] - R[0]) * (Z[2] - Z[0])
    -
    (R[2] - R[0]) * (Z[1] - Z[0]);

double area = 0.5 * std::fabs(detJ); // площадь треугольника.

double dLdr[3] = {
    (Z[1] - Z[2]) / detJ,
    (Z[2] - Z[0]) / detJ,
    (Z[0] - Z[1]) / detJ
};

double dLdz[3] = {
    (R[2] - R[1]) / detJ,
    (R[0] - R[2]) / detJ,
    (R[1] - R[0]) / detJ
};

double lamVertex[3];

for (int i = 0; i < 3; i++) {
    lamVertex[i] = lambda_f(tri.region, R[i], Z[i], t);
}

for (int q = 0; q < 7; q++) {
    double N[6];
    double dN_dL[6][3];

    getShapeFuncs(L_QUAD[q][0], L_QUAD[q][1], L_QUAD[q][2], N, dN_dL);

    double r = 0.0;
    double z = 0.0;

```

```

for (int i = 0; i < 3; i++) {
    r += L_QUAD[q][i] * R[i];
    z += L_QUAD[q][i] * Z[i];
}

double lam =
    L_QUAD[q][0] * lamVertex[0]
    +
    L_QUAD[q][1] * lamVertex[1]
    +
    L_QUAD[q][2] * lamVertex[2];

double gam = gamma_f(tri.region, r, z, t);
double fval = f_f(tri.region, r, z, t);

double weight = area * W_QUAD[q] * r;

double dNdr[6];
double dNdz[6];

for (int i = 0; i < 6; i++) {
    dNdr[i] =
        dN_dL[i][0] * dLdr[0]
        +
        dN_dL[i][1] * dLdr[1]
        +
        dN_dL[i][2] * dLdr[2];

    dNdz[i] =
        dN_dL[i][0] * dLdz[0]
        +
        dN_dL[i][1] * dLdz[1]
        +
        dN_dL[i][2] * dLdz[2];
}

for (int i = 0; i < 6; i++) {
    for (int j = 0; j <= i; j++) {
        double valG =
            (
                lam * (dNdr[i] * dNdr[j] + dNdz[i] * dNdz[j])

```

```

        +
        gam * N[i] * N[j]
    )
    * weight;

    double valM = N[i] * N[j] * weight;

    locG[i][j] += valG;
    locM[i][j] += valM;

    if (i != j) {
        locG[j][i] += valG;
        locM[j][i] += valM;
    }
}

locB[i] += fval * N[i] * weight;
}
}

addElement(G, rs, locG, locB, tri.n);

double zeroB[6] = { 0.0 };
addElement(M, rs, locM, zeroB, tri.n);
}
}

void applyNaturalBC(matrix &A, std::vector<double> &rs, double t)
{
    const double gPoints[3] = {
        0.5 - 0.3872983346207417,
        0.5,
        0.5 + 0.3872983346207417
    };

    const double gWeights[3] = {
        5.0 / 18.0,
        8.0 / 18.0,
        5.0 / 18.0
    };
};

```

```

auto integrateEdge = [&] (const Edge &e, int type) {
    int mid;

    if (!findEdgeMidNode(e.node1, e.node2, mid)) {
        return;
    }

    int idx[3] = {
        e.node1,
        mid,
        e.node2
    };

    double dr = nodes[e.node2].r - nodes[e.node1].r;
    double dz = nodes[e.node2].z - nodes[e.node1].z;

    double len = std::sqrt(dr * dr + dz * dz);

    double locM[3][3] = { 0.0 };
    double locB[3] = { 0.0 };

    for (int q = 0; q < 3; q++) {
        double s = gPoints[q];
        double w = gWeights[q];

        double phi[3] = {
            (1.0 - s) * (1.0 - 2.0 * s),
            4.0 * s * (1.0 - s),
            s * (2.0 * s - 1.0)
        };

        double r = 0.0;
        double z = 0.0;

        for (int i = 0; i < 3; i++) {
            r += phi[i] * nodes[idx[i]].r;
            z += phi[i] * nodes[idx[i]].z;
        }

        double weight = w * len * r;
    }

```



```

if (type == 2) {
    double theta = theta_s2(e.funcNum, r, z, t);

    for (int i = 0; i < 3; i++) {
        locB[i] += theta * phi[i] * weight;
    }
}
else {
    double beta = beta_s3(e.funcNum, r, z, t);
    double ub = u_beta_s3(e.funcNum, r, z, t);

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            locM[i][j] += beta * phi[i] * phi[j] * weight;
        }

        locB[i] += beta * ub * phi[i] * weight;
    }
}

for (int i = 0; i < 3; i++) {
    int row = idx[i];

    A.di[row] += locM[i][i];
    rs[row] += locB[i];

    for (int j = 0; j < i; j++) {
        int col = idx[j];

        int r_idx = row;
        int c_idx = col;

        if (r_idx < c_idx) {
            std::swap(r_idx, c_idx);
        }

        int pos = A.ig[r_idx];

        while (pos < A.ig[r_idx + 1] && A.jg[pos] != c_idx) {
            pos++;
        }
    }
}

```

```

        }

        if (pos < A.ig[r_idx + 1]) {
            A.ggl[pos] += locM[i][j];
            A.ggu[pos] += locM[j][i];
        }
    }
}

};

for (const auto &e : s2_edges) {
    integrateEdge(e, 2);
}

for (const auto &e : s3_edges) {
    integrateEdge(e, 3);
}
}

void applyDirichletBC(matrix &A, std::vector<double> &b, double t)
{
    for (const auto &bc : s1_nodes) {
        int idx = bc.node;

        double val = u_s1(bc.funcNum, nodes[idx].r, nodes[idx].z, t);

        A.di[idx] = 1.0;
        b[idx] = val;

        for (int k = A.ig[idx]; k < A.ig[idx + 1]; k++) {
            int j = A.jg[k];

            b[j] -= A.ggu[k] * val;

            A.ggl[k] = 0.0;
            A.ggu[k] = 0.0;
        }

        for (int i = idx + 1; i < A.n; i++) {
            for (int k = A.ig[i]; k < A.ig[i + 1]; k++) {
                if (A.jg[k] == idx) {

```

```

        b[i] -= A.ggl[k] * val;

        A.ggl[k] = 0.0;
        A.ggu[k] = 0.0;
    }
}
}
}
}

```

Tests.cpp

```

#include "header.h"
#include <cmath>

double u_exact(double r, double z, double t)
{
    return z * sin(t);
}

double lambda_f(int region, double r, double z, double t)
{
    return 1.0;
}

double gamma_f(int region, double r, double z, double t)
{
    return 0.0;
}

double f_f(int region, double r, double z, double t)
{
    return z * cos(t) - z * sin(t);
}

double u_s1(int funcNum, double r, double z, double t)
{
    return u_exact(r, z, t);
}

double theta_s2(int funcNum, double r, double z, double t)
{
    return 0.0;
}

double beta_s3(int funcNum, double r, double z, double t)
{
    return 0.0;
}

double u_beta_s3(int funcNum, double r, double z, double t)
{
    return 0.0;
}

double u0_f(double r, double z)
{
    return 0.0;
}

```

```
double v0_f(double r, double z)
{
    return z;
}

double a0_f(double r, double z)
{
    return 0.0;
}
```

LU.cpp

```
#include "header.h"
#include <cmath>
#include <iostream>

double scalar(const std::vector<double> &a, const std::vector<double> &b)
{
    double s = 0.0;

    for (size_t i = 0; i < a.size(); i++) {
        s += a[i] * b[i];
    }

    return s;
}

void multA(const matrix &A, const std::vector<double> &x, std::vector<double> &y)
{
    int n = A.n;
    y.assign(n, 0.0);

    for (int i = 0; i < n; i++) {
        y[i] += A.di[i] * x[i];

        for (int k = A.ig[i]; k < A.ig[i + 1]; k++) {
            int j = A.jg[k];

            y[i] += A.ggl[k] * x[j];
            y[j] += A.ggu[k] * x[i];
        }
    }
}

void calcLU(const matrix &A, matrix &LU)
{
    LU.n = A.n;
    LU.ig = A.ig;
    LU.jg = A.jg;

    LU.di.resize(A.n);
    LU.ggl = A.ggl;
    LU.ggu = A.ggu;

    for (int i = 0; i < A.n; i++) {
        double sumDi = 0.0;

        int i0 = LU.ig[i];
        int i1 = LU.ig[i + 1];

        for (int k = i0; k < i1; k++) {
            int j = LU.jg[k];
```

```

double sumL = 0.0;
double sumU = 0.0;

int p1 = i0;
int p2 = LU.ig[j];

while (p1 < k && p2 < LU.ig[j + 1]) {
    int c1 = LU.jg[p1];
    int c2 = LU.jg[p2];

    if (c1 == c2) {
        sumL += LU.ggl[p1] * LU.ggu[p2];
        sumU += LU.ggl[p2] * LU.ggu[p1];

        p1++;
        p2++;
    }
    else if (c1 < c2) {
        p1++;
    }
    else {
        p2++;
    }
}

LU.ggl[k] = (LU.ggl[k] - sumL) / LU.di[j];
LU.ggu[k] = LU.ggu[k] - sumU;

sumDi += LU.ggl[k] * LU.ggu[k];
}

LU.di[i] = A.di[i] - sumDi;

if (std::fabs(LU.di[i]) < 1e-15) {
    LU.di[i] = 1e-15;
}
}

void solveL(const matrix &LU, const std::vector<double> &b, std::vector<double> &y)
{
    y = b;

    for (int i = 0; i < LU.n; i++) {
        double sum = 0.0;

        for (int k = LU.ig[i]; k < LU.ig[i + 1]; k++) {
            sum += LU.ggl[k] * y[LU.jg[k]];
        }

        y[i] -= sum;
    }
}

void solveU(const matrix &LU, const std::vector<double> &y, std::vector<double> &x)
{
    x = y;

    for (int i = LU.n - 1; i >= 0; i--) {
        x[i] /= LU.di[i];

        for (int k = LU.ig[i]; k < LU.ig[i + 1]; k++) {
            int j = LU.jg[k];
            x[j] -= LU.ggu[k] * x[i];
        }
    }
}

```

```

    }
}

void doMSG_LU(const matrix &A, msg_lu &m, std::vector<double> &x, const
std::vector<double> &b, int maxIter, double eps)
{
    int n = A.n;

    matrix LU;
    calcLU(A, LU);

    m.r.resize(n);
    m.z.resize(n);
    m.t.resize(n);
    m.tmp.resize(n);
    m.x_tilde.resize(n);
    m.v1.resize(n);
    m.v2.resize(n);

    multA(A, x, m.tmp);

    for (int i = 0; i < n; i++) {
        m.r[i] = b[i] - m.tmp[i];
    }

    solveL(LU, m.r, m.tmp);
    solveU(LU, m.tmp, m.z);

    double rho_prev = scalar(m.r, m.z);

    double norm_b = scalar(b, b);
    if (std::fabs(norm_b) < 1e-20) {
        norm_b = 1.0;
    }

    double residual = std::sqrt(scalar(m.r, m.r) / norm_b);

    //std::cout << "Iter 0: Residual = " << residual << "\n";

    if (residual < eps || std::fabs(rho_prev) < 1e-30) {
        return;
    }

    std::vector<double> &p = m.v1;

    for (int i = 0; i < n; i++) {
        p[i] = m.z[i];
    }

    for (int k = 1; k <= maxIter; k++) {
        multA(A, p, m.t);

        double denom = scalar(p, m.t);

        if (std::fabs(denom) < 1e-30) {
            break;
        }

        double alpha = rho_prev / denom;

        for (int i = 0; i < n; i++) {
            x[i] += alpha * p[i];
            m.r[i] -= alpha * m.t[i];
        }
    }
}

```

```

    residual = std::sqrt(scalar(m.r, m.r) / norm_b);

    //if (k % 10 == 0 || residual < eps) {
    //    std::cout << "Iter " << k << ": Residual = " << residual << "\n";
    //}

    if (residual < eps) {
        break;
    }

    solveL(LU, m.r, m.tmp);
    solveU(LU, m.tmp, m.z);

    double rho_new = scalar(m.r, m.z);

    if (std::fabs(rho_prev) < 1e-30) {
        break;
    }

    double beta = rho_new / rho_prev;

    for (int i = 0; i < n; i++) {
        p[i] = m.z[i] + beta * p[i];
    }

    rho_prev = rho_new;
}
}

```

Header.h

```

#ifndef HEADER_H
#define HEADER_H

#include <vector>
#include <string>
using namespace std;

struct matrix {
    int n;
    vector<int> ig, jg;
    vector<double> di, ggl, ggu;
};

struct msg_lu {
    vector<double> r, z, t, tmp, x_tilde, v1, v2;
};

struct Node {
    double r, z;
};

struct Triangle {
    int n[6];
    int region;
};

struct S1Node {
    int node, funcNum;
};

struct Edge {
    int node1, node2, funcNum;
};

```

```

extern vector<Node> nodes;
extern vector<Triangle> triangles;
extern vector<S1Node> s1_nodes;
extern vector<Edge> s2_edges;
extern vector<Edge> s3_edges;

// функции теста
double lambda_f(int region, double r, double z, double t);
double gamma_f(int region, double r, double z, double t);
double f_f(int region, double r, double z, double t);

double u_s1(int funcNum, double r, double z, double t);
double theta_s2(int funcNum, double r, double z, double t);
double beta_s3(int funcNum, double r, double z, double t);
double u_beta_s3(int funcNum, double r, double z, double t);

double u_exact(double r, double z, double t);

// начальные условия
double u0_f(double r, double z);
double v0_f(double r, double z);
double a0_f(double r, double z);

// решатель
void multA(const matrix &A, const vector<double> &x, vector<double> &y);
void doMSG_LU(const matrix &A, msg_lu &m, vector<double> &x, const vector<double> &b,
int maxIter, double eps);

// МКЭ
void generatePortrait(matrix &A);
void assemble(matrix &G, matrix &M, vector<double> &rs, double t);
void applyNaturalBC(matrix &A, vector<double> &rs, double t);
void applyDirichletBC(matrix &A, vector<double> &b, double t);

void readData(int &maxIter, double &eps);

#endif

```